Compilador C--

Entrega 2: Analisador Sintático (Pág. 18)

João Victor Domingos e Souza, Nicole Garcia Montes Clemente,
Marcelo Americo da Silva, Davi Marques de Oliveira

18 de outubro de 2026

Sumário

1 Analisador léxico **1**

2 Analisador sintático **18**

3 Considerações finais **33**

Apêndices **35**

A Gramática Inicial **35**

B Gramática Transformada **36**

1 Analisador léxico

1.1 Visão geral Lexer

O projeto desenvolve um compilador C-- em quatro entregas. Esta primeira parte documenta a análise léxica; as próximas acrescentarão sintaxe, semântica e simulação ao mesmo documento. A organização segue os tópicos do enunciado (Disciplina de Compiladores, s.d.a, seção 1.1, págs. 1--2).

O programa recebe um arquivo `.cmm`, identifica tokens até EOF, consulta cada lexema válido em uma única tabela de símbolos e acumula diagnósticos com linha e coluna. A execução, a partir da raiz do projeto, é:

```
python main.py programa.cmm
```

São impressos nomes de tokens em `stdout`, um por linha, e erros em `stderr`. O retorno é 0 sem erros, 1 com erros léxicos e 2 nas falhas de entrada tratadas. A extensão `.cmm` é uma convenção do grupo.

A implementação usa Python 3.10 ou posterior, um AFD configurado em JSON e uma fachada <Lexer> que monta os componentes. A separação entre leitura, reconhecimento, armazenamento e apresentação mantém as responsabilidades dos componentes bem definidas e reduz o acoplamento entre eles. As interfaces entre fases se apoiam em (APPEL; GINSBURG, 1998, seção 1.1, págs. 4--5) e a responsabilidade do scanner em (AHO et al., 2007, seção 3.1.1, págs. 110--111). O texto inteiro é carregado em memória para permitir retrocesso com posição, linha e coluna; isso exige memória proporcional à entrada. <str>, listas e dicionários evitam limites fixos de comprimento.

A gramática C-- prevalece sobre C. A especificação N3220 do WG14, será usada para comparação das regras léxicas e ao longo do restante do desenvolvimento, e não como especificação integral da linguagem do trabalho (ISO/IEC JTC 1/SC 22/WG14, 2024, seção 6.4, pág. 52).

1.2 Tokens e Expressões Regulares

Tokens são categorias, padrões descrevem as sequências aceitas e lexemas são os textos encontrados na entrada, conforme (AHO et al., 2007, seção 3.1.2, pág. 111). A transformação da sequência de caracteres em tokens é apresentada por Appel e Ginsburg (APPEL; GINSBURG, 1998, cap. 2, págs. 16--38).

Neste trabalho, os tokens foram definidos a partir da gramática da linguagem C-- e das especificações fornecidas para o analisador. O reconhecimento é realizado através de um Autômato Finito Determinístico (AFD), cuja configuração é armazenada no arquivo `AFD.json` e carregada pela classe `AFD`.

A implementação utiliza expressões regulares somente para verificar algumas classes de caracteres durante as transições. A estrutura geral do reconhecimento continua sendo definida pelos estados e transições do AFD.

1.2.1 Padrões léxicos reconhecidos

Na implementação atual, as principais classes de lexemas reconhecidas são apresentadas na Tabela 1.

Tabela 1: Principais padrões léxicos reconhecidos pelo AFD.

Classe	Padrão ou regra	Exemplo	Estado final
Identificador	`[A-Za-z][A-Za-z0-9]*`	`x1`	`qID`
Número inteiro	`[0-9]+`	`32`	`qNUMINT`
Número decimal	`[0-9]+[.][0-9]+`	`12.50`	`qNUMFLOAT`
String	Texto entre aspas duplas	`"texto"`	`qSTRING_FINAL`
Caractere	Caractere ASCII entre aspas simples	`'a'`	`qCHAR_FINAL`

1.2.2 Identificadores

O padrão distingue maiúsculas de minúsculas e adapta (APPEL; GINSBURG, 1998, seção 2.2, Fig. 2.2, pág. 20), acrescentando maiúsculas. A letra inicial separa esse ramo do reconhecimento de números.

O reconhecimento de um identificador começa no estado inicial `q0`. Quando uma letra é encontrada, o AFD realiza uma transição para o estado `qID`.

A partir desse estado, novas letras ou dígitos podem continuar sendo consumidos:

```
q0  -- letra --> qID
qID -- letra | dígito --> qID
```

No método `_avaliarCondicao()`, as letras são reconhecidas por: [a-zA-Z] e a continuação do identificador utiliza: [a-zA-Z0-9]. São exemplos de identificadores reconhecidos:

```
x
x1
contador
variavel10
```

O estado `qID` pertence ao conjunto de estados finais do AFD. A distinção entre ID e palavra reservada é feita após consumir o lexema completo: `if` resulta em IF, enquanto `ifx` permanece ID. O procedimento e sua fundamentação estão descritos na Subseção 1.2.8.

1.2.3 Números

O AFD diferencia números inteiros de números de ponto flutuante. Quando um dígito é encontrado no estado inicial, ocorre a transição para o estado `qNUMINT`:

```
q0      -- dígito --> qNUMINT
qNUMINT -- dígito --> qNUMINT
```

Como `qNUMINT` é um estado final, são aceitos números como: 0, 10, 32, 1250.

Quando um ponto é encontrado depois da parte inteira, o autômato passa para o estado intermediário `qDOT`:

```
qNUMINT -- "." --> qDOT
```

O estado `qDOT` não é um estado final. Para concluir o reconhecimento de um número de ponto flutuante, pelo menos um dígito deve ser encontrado depois do ponto:

```
qDOT      -- dígito --> qNUMFLOAT
qNUMFLOAT -- dígito --> qNUMFLOAT
```

Dessa forma, são exemplos de números de ponto flutuante reconhecidos: 3.14, 10.5, 125.00. O ponto sozinho não é aceito como número, nem números sem dígitos após o ponto.

O predicado usa `re.fullmatch(r'[0-9]', caractere)` e admite apenas dígitos ASCII. Não há sinal, expoente ou sufixo no lexema numérico; o subconjunto reduz os estados desta entrega em relação à sintaxe completa de C (ISO/IEC JTC 1/SC 22/WG14, 2024, seções 6.4.4.1--6.4.4.3, págs. 56--63). A separação do sinal é coerente com N3220, seção 6.4.4.3, parágrafo 12, pág. 63. Os lexemas são armazenados como texto, sem conversão, opção descrita por (AHO et al., 2007, seção 3.1.3, Ex. 3.2, pág. 113).

1.2.4 Strings e caracteres

As strings começam quando uma aspa dupla é encontrada no estado inicial:

```
q0 -- " --> qSTRING
```

No estado `qSTRING`, os caracteres continuam sendo consumidos enquanto forem diferentes de uma nova aspa dupla e de uma quebra de linha. Quando a aspa dupla de fechamento é encontrada, o autômato passa para o estado final `qSTRING_FINAL`:

```
qSTRING -- " --> qSTRING_FINAL
```

Por exemplo:

```
"texto"
"ola mundo"
""
```

A string vazia é aceita pelo fechamento imediato: representa uma sequência de comprimento zero, como em C (ISO/IEC JTC 1/SC 22/WG14, 2024, seção 6.4.5, parágrafos 1 e 3, pág. 67).

O reconhecimento de caracteres utiliza uma sequência diferente:

```
q0     -- ' --> qCHAR1
qCHAR1 -- ASCII válido --> qCHAR2
qCHAR2 -- ' --> qCHAR_FINAL
```

Um exemplo é:

```
'a'
```

Na implementação atual, a condição `ASCII válido` é verificada através de:

```
len(caractere) == 1 and caractere.isascii()
    and caractere not in ("'", "\n", "\r")
```

Assim, o conteúdo deve ser uma única unidade ASCII, diferente de aspa simples, LF (*Line Feed*) e CR (*Carriage Return*). O espaço entre aspas simples é aceito; constante vazia e três aspas simples são rejeitadas. A gramática exige uma unidade ASCII; em C, o conteúdo da constante também não é opcional e exclui aspa bruta e quebra de linha (ISO/IEC JTC 1/SC 22/WG14, 2024, seção 6.4.4.5, parágrafos 1--3, págs. 63--64). A restrição local a uma unidade difere das constantes multicaractere admitidas por C, cujo valor depende da implementação (ISO/IEC JTC 1/SC 22/WG14, 2024, seção 6.4.4.5, parágrafo 11, pág. 65).

A barra invertida é conteúdo comum quando o caminho permite. `'\n'` é rejeitado por conter dois caracteres; `"\n"` preserva barra e letra. Não interpretar escapes é uma simplificação no projeto, que não afeta a análise léxica. A gramática de C-- não define escapes, portanto a implementação atual é consistente com a sintaxe da linguagem mas diferente de C (ISO/IEC JTC 1/SC 22/WG14, 2024, seção 6.4.4.5, parágrafos 3--4, págs. 64--65). Uma melhoria possível seria permitir escapes.

1.2.5 Operadores

Os operadores possuem caminhos próprios no AFD. Alguns são reconhecidos com apenas um caractere, enquanto outros dependem da leitura de um segundo caractere. A Tabela 2 apresenta os operadores implementados.

Tabela 2: Operadores reconhecidos pelo AFD.

Lexema	Estado final	Token
+	qPLUS	PLUS
++	qINC	INCREMENT
-	qMINUS	MINUS
-	qDEC	DECREMENT
*	qMULT	MULT
/	qDIV	DIV
%	qMOD	MOD
=	qASSIGN	ASSIGN
==	qEQ	EQUALS
<	qLT	LESS_THAN
<=	qLEQ	LEQ
>	qGT	GREATER_THAN
>=	qGEQ	GEQ
!	qNOT	NEG
!=	qNEQ	DIFF
&&	qAND	AND
||	qOR	OR

Nos casos em que um operador pode possuir um ou dois caracteres, o primeiro estado já pode representar um lexema válido.

Por exemplo:

```
q0    -- + --> qPLUS
qPLUS -- + --> qINC
```

Assim, + e ++ podem ser reconhecidos separadamente. A mesma ideia é utilizada para os pares -/-, =/==, </<=, >/>= e !/!=.

1.2.6 Símbolos de pontuação

Cada símbolo de pontuação possui um caminho próprio a partir de q0. Os símbolos reconhecidos são apresentados na Tabela 3.

Tabela 3: Símbolos de pontuação reconhecidos pelo AFD.

Lexema	Estado final	Token
(	qLPARENT	LPARENT
)	qRPARENT	RPARENT
[	qLBRACKET	OPEN_BRACKETS
]	qRBRACKET	CLOSE_BRACKETS
{	qLBRACE	OPEN_BRACES
}	qRBRACE	CLOSE_BRACES
,	qCOMMA	COMMA
;	qSEMICOLON	SEMICOLON

A utilização de tokens separados para cada operador e símbolo de pontuação segue a forma recomendada nas especificações do trabalho.

1.2.7 Comentários e espaços em branco

Os comentários aceitos pela linguagem possuem o formato: `/* comentario */`. O reconhecimento começa pelo mesmo caractere utilizado pelo operador de divisão. Inicialmente, `/` leva o autômato ao estado `qDIV`. Caso o caractere seguinte seja `*`, o AFD segue para o caminho de comentário:

```
q0           -- / --> qDIV
qDIV         -- * --> qCOMMENT
qCOMMENT     -- * --> qCOMMENTSTAR
qCOMMENTSTAR -- / --> qCOMMENTDONE
```

O estado final `qCOMMENTDONE` indica um comentário completo, que é descartado sem gerar token por repetição iterativa do reconhecimento. Não há aninhamento, conforme a delimitação de C (ISO/IEC JTC 1/SC 22/WG14, 2024, seção 6.4.9, parágrafo 1 e nota 80, pág. 70). `//` não inicia comentário em C--, embora seja permitido em C. Espaços dentro de strings são preservados. Se EOF ocorrer antes de `*/`, o lexema completo produz ‘‘Comentário não terminado’’.

Os espaços em branco também não geram tokens. No estado inicial, a condição de `whitespace` mantém o autômato no próprio `q0`:

```
q0 -- whitespace --> q0
```

Na classe `AFD`, essa condição verifica explicitamente o conjunto formado por espaço comum, tabulação horizontal, LF e CR. <Reconhecedor._ignorar_espacos()> usa o mesmo conjunto. O descarte de espaços e comentários entre tokens é uma operação comum durante a análise léxica e também é discutido por Appel e Ginsburg (APPEL; GINSBURG, 1998, cap. 2, págs. 16--38).

1.2.8 Palavras reservadas

As palavras reservadas não possuem estados próprios no AFD. Elas seguem inicialmente o mesmo caminho utilizado pelos identificadores e chegam ao estado `qID`. Depois que o lexema completo é reconhecido, ele é consultado na tabela de símbolos. Caso esteja cadastrado como palavra reservada, seu token específico é utilizado. Caso contrário, permanece classificado como `ID`.

Esse procedimento segue a estratégia apresentada por Aho et al. (AHO et al., 2007, seção 3.4.2, págs. 132--133).

As palavras reservadas previstas no projeto são apresentadas na Tabela 4.

Tabela 4: Palavras reservadas definidas para a linguagem.

Lexema	**Token**
`int`	INT
`float`	FLOAT
`char`	CHAR
`bool`	BOOL
`typedef`	TYPEDEF
`struct`	STRUCT
`if`	IF
`else`	ELSE
`while`	WHILE
`readln`	READLN
`print`	PRINT
`break`	BREAK
`return`	RETURN
`true`	TRUE
`false`	FALSE

1.2.9 Valores dos tokens

Os tipos pertencem a um <Enum> com <auto()>; o valor inteiro de cada membro está disponível em <tipo.value>. Esses valores pertencem à enumeração utilizada pelo projeto e são independentes dos números utilizados para identificar os estados do AFD.

Por exemplo, o número de um estado dentro do arquivo `AFD.json` serve para identificar uma posição do autômato, enquanto o valor de um token representa sua categoria dentro do restante do compilador.

A Tabela 5 apresentam os 47 valores utilizados pelo projeto.

Tabela 5: Tokens e valores inteiros utilizados pelo analisador.

Valor	Token	Exemplo
1	ID	x1
2	NUMINT	32
3	NUMFLOAT	12.50
4	LITERAL	"abc"
5	ASCII	'a'
6	COMMA	,
7	OPEN_BRACKETS	[
8	CLOSE_BRACKETS	]
9	LPARENT	(
10	RPARENT	)
11	OPEN_BRACES	{
12	CLOSE_BRACES	}
13	PLUS	+
14	MINUS	-
15	DECREMENT	--
16	INCREMENT	++
17	NEG	!
18	ASSIGN	=
19	EQUALS	==
20	LESS_THAN	<
21	LEQ	<=
22	GREATER_THAN	>
23	GEQ	>=
24	DIFF	!=
25	OR	||
26	AND	&&
27	MULT	*
28	DIV	/
29	MOD	%
30	SEMICOLON	;
31	INT	int
32	FLOAT	float
33	CHAR	char
34	BOOL	bool
35	TYPEDEF	typedef
36	STRUCT	struct
37	IF	if
38	ELSE	else
39	WHILE	while
40	READLN	readln
41	PRINT	print
42	BREAK	break
43	RETURN	return
44	TRUE	true
45	FALSE	false
46	EOF	fim da entrada
47	UNKNOWN	erro interno

1.2.10 Lexemas e atributos dos tokens

Além do seu tipo, cada token pode possuir informações relacionadas ao lexema reconhecido e à posição em que ele foi encontrado. (AHO et al., 2007, seção 3.1.3, págs. 112--113) apresentam o uso de atributos para associar informações adicionais aos tokens reconhecidos pelo analisador. Na implementação deste trabalho são utilizadas informações como:

- `tipo`: identifica a categoria do token;
- `lexema`: armazena a sequência de caracteres reconhecida;
- `linha`: indica a linha em que o lexema foi encontrado;
- `coluna`: indica sua posição inicial na linha;
- `indiceSimbolo`: associa cada token com lexema à entrada correspondente na tabela de símbolos; EOF não possui índice.

A Tabela 6 apresenta um exemplo para a entrada:

```
if(x1 <= 32) b = 10;
```

Tabela 6: Exemplos de tokens, lexemas e atributos.

Token	Lexema	Linha	Coluna	Índice
IF	if	1	1	4
LPARENT	(	1	3	18
ID	x1	1	4	40
LEQ	<=	1	7	30
NUMINT	32	1	10	41
RPARENT	)	1	12	19
ID	b	1	14	42
ASSIGN	=	1	16	27
NUMINT	10	1	18	43
SEMICOLON	;	1	20	39

Nesse exemplo, todos os tokens correspondentes a lexemas reconhecidos possuem uma referência para a tabela de símbolos. Identificadores e literais novos são inseridos durante a análise; palavras reservadas, operadores e símbolos de pontuação reutilizam entradas previamente carregadas.

1.2.11 Representação dos tokens na saída

Cada token é apresentado por seu nome na saída normal do programa; os valores inteiros não são impressos. Para a entrada da Tabela 6, a sequência produzida, com um token por linha, é:

```
IF
LPARENT
ID.x1
LEQ
NUMINT.32
RPARENT
ID.b
ASSIGN
NUMINT.10
SEMICOLON
EOF
```

Nos casos de identificadores e números, o lexema aparece junto ao nome do token. LITERAL e ASCII preservam o lexema no objeto, mas imprimem somente o nome da categoria.

1.2.12 AFD para reconhecimento dos tokens

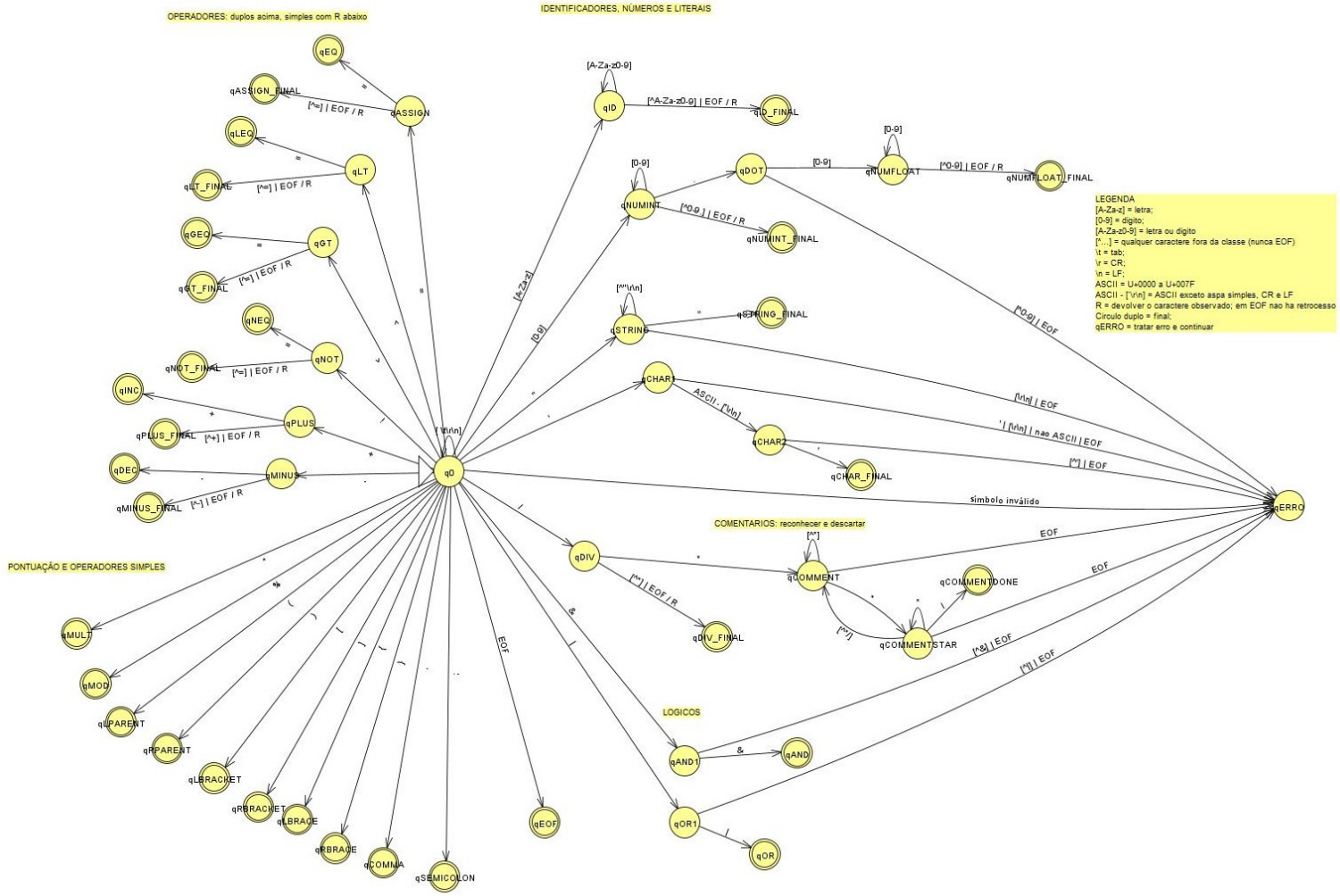


Figura 1: AFD projetado para o analisador léxico de C--.

A Figura 1 reúne em um AFD os caminhos de identificadores, números, literais, operadores (inclusive lógicos), pontuação e comentários, agrupados para facilitar a leitura. Círculos duplos indicam aceite. O reconhecimento por autômatos segue Appel e Ginsburg (APPEL; GINSBURG, 1998, seção 2.3, págs. 21--23) e (AHO et al., 2007, seção 3.4, págs. 128--136).

1.2.13 Relação entre o AFD projetado e a implementação

O diagrama da Figura 1 é a referência visual do AFD projetado. Ele explicita estados auxiliares de finalização, como os terminados em `_FINAL`, o estado `qEOF` e as arestas marcadas com `R`, que devolvem ao fluxo de entrada o caractere que pertence ao próximo token.

A configuração executável em `AFD.json` codifica a mesma estratégia de reconhecimento com uma representação operacional mais compacta: estados como `qID`, `qNUMINT` e `qASSIGN` já pertencem ao conjunto de aceitação; o reconhecedor memoriza o último aceite e restaura o marco do leitor quando necessário; e o EOF é produzido diretamente pelo controle da entrada. Por isso, o desenho e o JSON devem ser comparados pelo comportamento léxico, e não por uma correspondência individual entre nomes e quantidade de estados. O JSON executado possui 41 estados e 31 finais; cada estado tem número e nome. O estado inicial é definido por:

```
"initial_state": 0
```

que corresponde ao estado `q0`. Os estados considerados de aceitação são armazenados separadamente em:

```
"final_states": [...]
```

A classe `AFD` carrega essa informação e verifica se o estado atual é final através do método:

```
def ehEstadoFinal(self):
    return self.estadoAtual in self.estadosFinais
```

No JSON, `qID`, `qNUMINT`, `qNUMFLOAT` e `qASSIGN` pertencem diretamente ao conjunto de finais. O reconhecedor memoriza o último aceite para decidir a fronteira pelo maior lexema e restaura o cursor quando o caractere observado pertence ao token seguinte.

As transições também são armazenadas no JSON. Cada uma delas possui:

- o estado de origem;
- o estado de destino;
- a condição que deve ser satisfeita pelo caractere lido.

Por exemplo:

```
{"from": 0, "to": 1, "read": "letra"}
```

representa a transição:

```
q0 -- letra --> qID
```

Já:

```
{"from": 2, "to": 3, "read": "."}
```

representa a passagem de um número inteiro para o estado que aguarda a parte decimal.

O método `transicionar()` percorre as transições disponíveis para o estado atual e utiliza `_avaliarCondicao()` para verificar qual delas aceita o caractere recebido.

Quando existe uma lista de transições para o estado atual e nenhuma condição é satisfeita, o método move o autômato para o estado cujo nome é `qERRO` e retorna falso. Se o estado nem possui lista de transições, retorna falso sem mudá-lo. Em ambos os casos, o reconhecedor consulta o último aceite. Na configuração atual, o estado qERRO tem número 40:

Assim, o arquivo `AFD.json` fornece a estrutura operacional utilizada diretamente pelo programa, enquanto a Figura 1 registra o AFD de referência solicitado para o projeto. Os rótulos de EOF do JSON não são avaliados como eventos de fim de arquivo: o reconhecedor produz EOF diretamente, como explicado a seguir.

1.2.14 Fim do arquivo

O fim do arquivo é representado pelo token `EOF`, cujo valor inteiro é 46. Esse token possui uma função de controle e informa às demais partes do programa que todos os caracteres da entrada já foram processados.

Diferentemente dos demais lexemas, o `EOF` não corresponde a uma sequência escrita pelo usuário no arquivo fonte. Ele é produzido quando o analisador identifica que a entrada chegou ao fim.

1.3 Implementação

1.3.1 Entrada, saída e arquitetura

A Tabela 7 apresenta as classes e responsabilidades do analisador léxico, que são detalhadas nas seções seguintes.

Tabela 7: Classes e responsabilidades no Lexer.

Arquivo/classe	Responsabilidade efetivamente implementada
<main.py>, <src/Lexer.py>	Recebe argumentos e apresenta resultados; Valida o arquivo, recria tabela/erros e monta os colaboradores.
<src/core/LeitorTexto.py>	Carrega UTF-8 em memória; observa/avança caracteres e salva/restaura posição, linha e coluna.
<src/AFD.py> e <AFD.json>	Carregam estados, finais e listas de transições; avaliam condições por literais e predicados.
<src/core/Reconhecedor.py>	Encontra o maior lexema, descarta comentários iterativamente e agrupa construções incompletas para diagnóstico.
<src/core/MotorLexer.py>	Solicita tokens até EOF, consulta todos os lexemas na tabela única e encaminha erros ao gerenciador.
<src/TabelaSimbolos.py>, <GerenciadorErros.py>	Mantêm o dicionário de símbolos e a lista de diagnósticos.
<src/models/> e <TiposToken.py>	Definem registros de token, símbolo, erro e a enumeração dos tipos.

A Figura 2 apresenta o diagrama de classes do analisador léxico, com os módulos e classes do projeto. As setas indicam dependências entre classes. A interface de execução estão na Tabela 7.

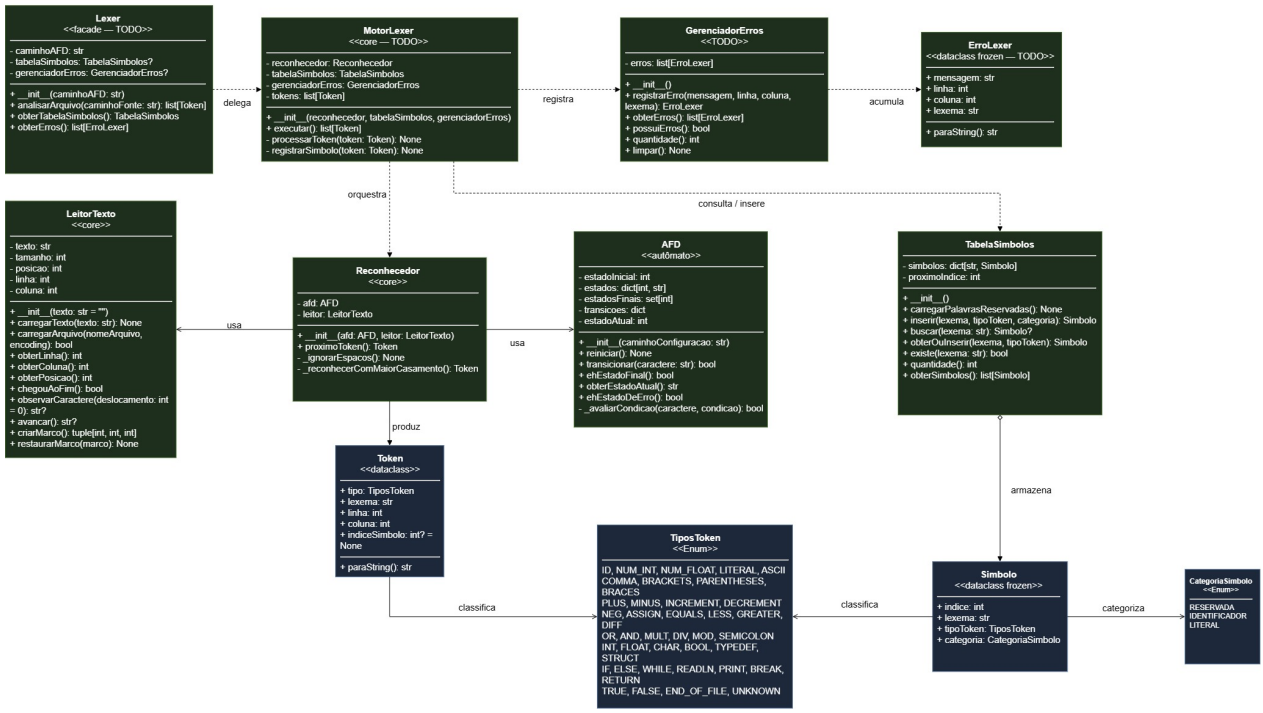


Figura 2: Diagrama de classes do analisador léxico.

A fachada cria uma tabela de símbolos e um gerenciador de erros por análise, enquanto reconhecedor e motor recebem somente as dependências necessárias. <Simbolo> é imutável; <Token> recebe o índice resolvido antes de ser devolvido. Essa divisão mantém leitura, reconhecimento, armazenamento e apresentação em componentes distintos.

As regras, exemplos e justificativas de cada classe de token estão na Subseção 1.2.

1.3.2 Algoritmo de reconhecimento e retrocesso

Estruturas e métodos. A configuração de estados, finais e transições e o método <ehEstadoFinal()> estão descritos na Subseção 1.2.13. Em memória, <AFD> usa dicionário de estados, conjunto de finais e listas de transições por origem. <reiniciar()> volta a `q0`; <_avaliarCondicao()> interpreta predicados conhecidos, sem executar expressões regulares arbitrárias do JSON. Essa representação segue (APPEL; GINSBURG, 1998, seção 2.3, págs. 21--23).

Algoritmo. <Reconhecedor.proximoToken()> implementa o maior lexema com memória do último estado final, como descrito por (APPEL; GINSBURG, 1998, seção 2.3, pág. 23) e (AHO et al., 2007, seção 3.4, págs. 128--136):

1. Reinicia o AFD, ignora separadores e retorna EOF se a entrada acabou.
2. Salva o início e avança enquanto há transição válida. A cada estado final, salva o lexema e o marco de posição, linha e coluna.
3. Se encontrou um final, restaura seu marco e classifica pelo mapa <ESTADO_PARA_TOKEN>.
4. Se reconheceu `qCOMMENTDONE`, descarta o comentário e reinicia o laço, sem recursão.
5. Se string, caractere, decimal ou comentário ficaram incompletos, agrupa o lexema inválido e retorna UNKNOWN com mensagem específica.
6. Nos demais casos sem final, restaura o início, consome um caractere e retorna UNKNOWN, garantindo progresso.

<MotorLexer.executar()> repete a solicitação até EOF; <processarToken()> encaminha UNKNOWN ao gerenciador e consulta cada lexema válido na tabela, inclusive reservadas, operadores e

delimitadores. <LeitorTexto.criarMarco()> e <restaurarMarco()> salvam/restauram uma tupla de três inteiros. O armazenamento integral substitui o esquema de dois buffers de (AHO et al., 2007, seção 3.2, págs. 115--117); favorece a simplicidade, com custo de memória proporcional ao arquivo.

Fronteira do token. Em `=x`, o reconhecedor salva o aceite de `=` e retorna ASSIGN, deixando `x` para a próxima chamada. A relação entre o algoritmo, a codificação em JSON e o desenho está na Subseção 1.2.13. (Disciplina de Compiladores, s.d.a, pág. 1).

1.4 Tabela de símbolos

A classe <TabelaSimbolos> mantém um único `dict[str, Simbolo]` compartilhado durante cada análise. A chave é o lexema completo; o valor imutável reúne índice, lexema, tipo de token e categoria. Dicionários fornecem busca associativa, conforme a implementação por hash discutida por (APPEL; GINSBURG, 1998, seção 5.1, págs. 106--109). A consulta prévia das palavras reservadas segue (AHO et al., 2007, seção 3.4.2, págs. 132--133).

1.4.1 Categorias e conteúdo inicial

<CategoriaSimbolo> diferencia quatro grupos: RESERVADA, IDENTIFICADOR, LITERAL e FIXO. O último reúne operadores e delimitadores; ele permite cumprir a exigência de pesquisar toda string reconhecida antes de devolver o token (Disciplina de Compiladores, s.d.a, pág. 3).

O construtor chama <carregarTokensIniciais()>. Primeiro insere as 15 palavras reservadas da Tabela 4; depois insere os 25 lexemas fixos, como `+`, `<=`, parênteses e `;`. Cada entrada recebe um índice crescente e estável na análise.

1.4.2 Operações e algoritmo

Tabela 8: Interface de <TabelaSimbolos>.

Método	Contrato
<carregarTokensIniciais()>	Insere reservadas e lexemas fixos com seus tipos e categorias.
<inserir(lexema,tipoToken, categoria)>	Reutiliza a entrada existente ou cria uma nova com o próximo índice.
<buscar(lexema)>	Retorna o <Simbolo> associado ou <None>.
<obterOuInserir(lexema, tipoToken)>	Consulta todo lexema; se novo, classifica-o como identificador, literal ou fixo e insere.
<existe(lexema)>	Informa se o lexema já está armazenado.
<remover(lexema)>	Remove uma entrada e informa se ela existia; não renumera as demais.
<quantidade()>	Retorna o número total de entradas.

O motor chama <obterOuInserir()> para todo token válido, com exceção de EOF, que não possui lexema. Se a entrada já existe, seu tipo prevalece: assim, o AFD reconhece `if` inicialmente pelo padrão de ID, mas a tabela o resolve como <TiposToken.IF>. Identificadores e literais novos são cadastrados, e o índice encontrado é copiado para <Token.indiceSimbolo>. Operadores e delimitadores encontram suas entradas previamente carregadas. Essa referência por índice corresponde ao atributo de token descrito por (AHO et al., 2007, seção 3.1.3, págs. 112--113).

Ocorrências repetidas reutilizam o mesmo registro. Em uma análise de `x x "a" "a"`, os dois IDs compartilham um índice, as duas strings compartilham outro e a tabela termina com 42 entradas: 40 iniciais e duas dinâmicas. Uma nova chamada à fachada recria a tabela, impedindo vazamento de símbolos entre arquivos. Escopos e declarações pertencem às fases sintática e semântica, não à classificação léxica (AHO et al., 2007, seção 2.7, pág. 86).

1.5 Gerenciador de erros léxicos

O gerenciador acumula diagnósticos sem interromper a análise. A recuperação por descarte controlado se apoia em (AHO et al., 2007, seção 3.1.4, págs. 113--114) e (APPEL; GINSBURG, 1998, seção 2.2, Fig. 2.2, pág. 20).

1.5.1 Estrutura e interface

<GerenciadorErros.erros> é uma lista de registros imutáveis <ErroLexer>, cada um com mensagem, linha, coluna e lexema. A contagem deriva do tamanho da lista, sem estado redundante.

Tabela 9: Interface do gerenciador de erros.

Método	Contrato
<registrarErro(mensagem,linha,coluna, lexema)>	Cria, armazena e retorna um diagnóstico.
<registrarCaractereInvalido(caractere,linha, coluna)>	Atalho para diagnosticar um caractere inválido.
<possuiErros()>	Informa se algum erro foi registrado.
<quantidade()>	Retorna a quantidade de erros.
<obterErros()>	Retorna uma cópia da lista.
<limpar()>	Remove os diagnósticos anteriores.

1.5.2 Detecção, recuperação e apresentação

O reconhecedor associa uma mensagem ao marcador UNKNOWN. Para um caractere isolado inválido, restaura o marco inicial, consome exatamente uma unidade e usa ‘‘Token não reconhecido’’. Para construções iniciadas que não podem terminar validamente, consome o fragmento como uma unidade e informa: ‘‘Número decimal incompleto’’, ‘‘Literal de string não terminado’’, ‘‘Literal de caractere inválido ou não terminado’’ ou ‘‘Comentário não terminado’’.

O motor registra o erro, omite UNKNOWN da sequência e solicita o token seguinte. Quebras de linha não são consumidas junto com strings ou caracteres malformados, preservando a posição da próxima tentativa. Linhas e colunas começam em 1 e CRLF conta como uma única mudança de linha. Para `"aberto` na linha 3, coluna 8, o formato apresentado é:

```
Erro léxico na linha 3, coluna 8: Literal de string não terminado.
Lexema: '"aberto'
```

<main.py> imprime somente tokens válidos, os diagnósticos e retorna código 1 quando há erro léxico. O processo continua até EOF, permitindo relatar múltiplos problemas em uma única execução.

1.6 Resultados experimentais

A verificação automatizada executa a fachada real e compara tokens, diagnósticos e posições com expectativas explícitas. O programa empregado no teste principal é apresentado integralmente a seguir. Ele reúne palavras reservadas, identificadores, literais, todos os grupos de operadores, delimitadores, comentário multilinha e três erros léxicos intencionais.

Programa 1: Programa utilizado no teste principal.

```
/* Teste */
typedef struct {
    int codigo;
    float saldo;
    char categoria;
    bool ativo;
} Conta;

int calcular(int a, int b) {
    int resultado;
    resultado = (a + b) * 2 / 1 % 3 - 1;
    return resultado;
}

int main() {
    Conta contas[10];
    int indice;
    float valor;
    char letra;
    bool valido;
    int x, y;

    x+++++y;
    x++ + ++y;

    indice = 0;
    valor = 12.50;
    letra = 'A';
    valido = true;

    /* O comentario pode ocupar
       mais de uma linha. */
    while (indice < 10 && valido == true) {
        readln(valor);

        if (valor >= 100.0 || valor != 0.0) {
            print("valor valido");
            indice++;
        } else {
            if (valor <= 0.0 && indice > 0) {
                indice--;
            }

            valido = !false;
            break;
        }

        /*erros*/
        float valor@base = 10.5; /* O caractere @ nao e reconhecido pelo lexer. */
        print("valor valido);
        valor = 12.50.8;
    }

    return calcular(indice, 2);
}
```

A execução percorreu todo o programa, produziu 192 tokens válidos, incluindo EOF, e terminou com código 1 devido aos três erros esperados. O caractere `@` foi diagnosticado na linha 49, coluna 20; a string sem fechamento, na linha 50, coluna 15; e o segundo ponto da construção `12.50.8`, na linha 51, coluna 22. Mesmo após cada ocorrência, o analisador continuou até o fim da entrada. Os casos exercitam maior lexema e recuperação, discutidos por (APPEL; GINSBURG, 1998, seções 2.2--2.3, págs. 20--23) e (AHO et al., 2007, seção 3.1.4, pág. 114).

Tabela 10: Entradas e saídas verificadas; erros contados separadamente.

Entrada	Sequência de saída	Erros
`if ifx`	`IF ID.ifx EOF`	0
`32 12.50`	`NUMINT.32 NUMFLOAT.12.50 EOF`	0
`= ==`	`ASSIGN EQUALS EOF`	0
`' '`	`ASCII EOF`	0
`""`	`LITERAL EOF`	0
`/* ok */x`	`ID.x EOF`	0
`''`	`EOF`	1
`'''`	`EOF`	2
`U+0661 U+0662`	`EOF`	2
`@\nx`	`ID.x EOF`	1
`12.`	`EOF`	1
`"abc`	`EOF`	1
`/* aberto`	`EOF`	1

Todos os casos da tabela corresponderam às expectativas. Comentário, string, caractere e decimal incompletos produzem um diagnóstico específico por construção (três aspas simples formam duas construções inválidas). O caso com `@` comprova a continuidade e as posições (1,1) para o erro e (2,1) para o ID seguinte. EOF foi verificado em toda entrada.

Também passaram as verificações de identificador e inteiro com 10.000 caracteres, reutilização de índices, 1.500 comentários consecutivos sem recursão e o exemplo do enunciado.

2 Analisador sintático

2.1 Visão geral e organização

O analisador sintático recebe os tokens da primeira etapa, reconhece as construções de C-- e monta uma árvore de sintaxe abstrata (AST) em memória. Foi implementado em Python como parser recursivo descendente preditivo: os métodos correspondem aos não terminais e escolhem produções pelo token corrente. Essa organização permite acompanhar a derivação e associar a criação dos nós às produções. A fundamentação utiliza a análise preditiva de (APPEL; GINSBURG, 1998, seção 3.2), as transformações e a análise descendente de (AHO et al., 2007, seções 4.3--4.4) e os requisitos da segunda entrega (Disciplina de Compiladores, s.d.b).

A execução a partir da raiz do projeto é:

```
python main.py arquivo.cmm --arvore --imagem "arvore.png"
```

2.1.1 Fundamentação do método e das interfaces

A correspondência entre um não terminal e um procedimento é descrita em (LOUDEN, 2004, seção 4.1.1, p. 144--145): terminais tornam-se chamadas de casamento, não terminais tornam-se chamadas de procedimentos e alternativas tornam-se decisões condicionais. Por isso, `MotorParser` conserva métodos reconhecíveis a partir da gramática, enquanto `casar` concentra comparação e avanço. A recursão representa naturalmente o aninhamento de expressões e comandos; o token corrente permite decidir a alternativa antes de descer. Optou-se pela implementação manual porque ela permite posicionar explicitamente as ações de construção e recuperação, flexibilidade discutida por (LOUDEN, 2004, seção 4.1.2, p. 150--151). Isso facilita conferir o rastro.

A separação dos módulos adapta o princípio de interfaces entre fases de (APPEL; GINSBURG, 1998, seção 1.1, p. 4--5). `FluxoTokens` encapsula a posição de leitura para que os reconhecedores compartilhem um único estado de entrada; `ConjuntosLL1` reúne as decisões preditivas para evitar conjuntos divergentes entre métodos. `Parser` fornece ao restante do compilador uma operação que retorna a raiz e uma consulta de diagnósticos. Esses nomes de classes e a distribuição em arquivos são escolhas do projeto para concretizar os princípios citados, e não uma arquitetura prescrita literalmente pelos livros. A Figura 3 apresenta o diagrama de classes proposto para o Parser. A interface de execução estão na Tabela 11.

Tabela 11: Classes e responsabilidades no Parser.

Módulo	Responsabilidade e estruturas principais
<Parser.py>	Fachada que prepara o fluxo e o motor, limpa os erros de uma análise anterior e devolve a raiz `Programa`.
<core/FluxoTokens.py>	Lista de tokens e posição corrente; consulta, avanço, verificação de tipos e reconhecimento do fim.
<core/ConjuntosLL1.py>	Conjuntos de início, FIRST/FOLLOW usados pelo motor, não terminais anuláveis e seguidores admitidos para inserção de delimitadores.
<core/MotorParser.py>	Métodos recursivos de reconhecimento, casamento de terminais, recuperação e composição da AST.
<ErroSintatico.py> e <GerenciadorErrosSintaticos.py>	Registro de mensagem, token encontrado, esperados e recuperação; armazenamento em lista e emissão configurável.
<SaidaParser.py>	Emissão do nome do não terminal visitado e de cada terminal efetivamente casado.
<ast/Nos.py>	Classes de dados independentes do motor, com atributos e referências aos filhos.
<VisualizadorArvore.py>	Percurso dos nós e listas para impressão textual; exportação opcional com Graphviz.

O motor consulta o fluxo e os conjuntos preditivos, usa os serviços de saída e erros e cria diretamente os nós da árvore. A fachada devolve a raiz ao chamador, que pode solicitar a visualização

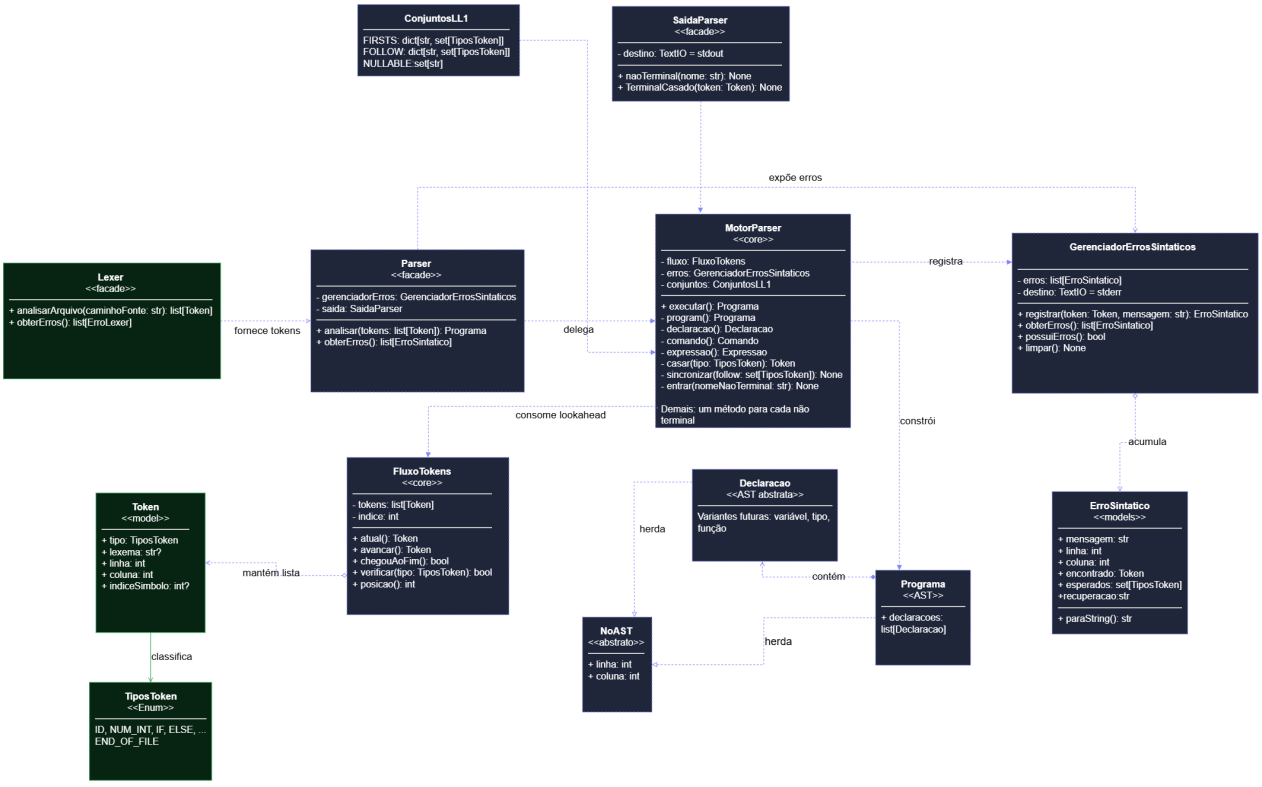


Figura 3: Diagrama de Classes do analisador sintático.

2.2 Da gramática inicial à gramática LL(1)

A gramatica inicial é aprsentada no Apêndice A e a gramática transformada usada na verificação é apresentada no Apêndice B. Foram feitas transformações iniciais para eliminação de recursão à esquerda, e após isso, fatoração e organização por precedência (AHO et al., 2007, seções 4.3.3-

-4.3.4 e 4.4), análise preditiva (APPEL; GINSBURG, 1998, seção 3.2) e a ordem de operadores solicitada (Disciplina de Compiladores, s.d.b).

A única mudança realizada nas regras da linguagem inicial é permitir blocos vazios, inclusive corpos de funções. As demais alterações reorganizam as produções para decisão preditiva e explicitam a precedência exigida. A aceitação de uma expressão no lado esquerdo da atribuição já decorre de `Expr = Expr` e segue a orientação do enunciado de adiar a validação de l-values.

2.2.1 Normalização da notação

Foram uniformizados os nomes de produções e algumas grafias, como typedef. Os terminais foram relacionados aos tokens do léxico: números em expressões usam `NUM_INT` ou `NUM_FLOAT`; o tamanho de vetor usa `NUM_INT`; caracteres usam o token `ASCII`, que já contém o literal delimitado. Essas correspondências documentam a representação dos tokens utilizada pelo programa.

2.2.2 Listas e prefixos comuns

A recursão à esquerda de identificadores foi substituída por uma cauda. Uma tradução direta de `IdList -> IdList , ID Array` chamaria o mesmo procedimento antes de consumir qualquer token, impedindo o avanço. Aplicou-se o esquema de (LOUDEN, 2004, seção 4.2.3, p. 158): de

$A \to A\alpha \mid \beta$

para

$A \to \beta A'$ e $A' \to \alpha A' \mid \varepsilon$.

Neste caso, β é `ID Array` e α é `, ID Array`; ambos os esquemas descrevem um primeiro elemento seguido de zero ou mais repetições:

```
Inicial: IdList -> ID Array | IdList , ID Array
Final: idList -> ID Array idList'
        idList' -> , ID Array idList' | epsilon
```

Cada chamada produz declarações na ordem da entrada. Listas de argumentos também foram fatoradas: `ExprList'` reconhece uma expressão, e `RestoExprList` decide entre vírgula seguida de outra expressão e a produção vazia. Parâmetros mantêm a estrutura `FormalList/FormalRest`.

Declarações globais de variável e função começam por `Type ID`. Esse prefixo passou a ser consumido uma única vez em `Program`; `RestoDecl` escolhe a função quando encontra `(`, ou continua a variável por `Array idList' ;`. A repetição de estruturas foi deslocada para `Program`, e `TypeDeclOne` reconhece uma estrutura por chamada. Isso evita alternativas anuláveis concorrentes no nível do programa. A fatoração usa o princípio

$A \to \alpha\beta \mid \alpha\gamma$

transformado em

$A \to \alpha A'$

$A' \to \beta \mid \gamma$,

apresentado em (LOUDEN, 2004, seção 4.2.3, p. 162--163). O prefixo comum é reconhecido antes da escolha; assim, não é necessário tentar uma declaração, voltar na entrada e tentar outra.

A terminação também foi preservada: na gramática inicial a alternativa final de `Program` é uma função. Na final, somente o ramo de função alcança `Program' -> epsilon`; os ramos de variável e estrutura exigem outro `Program`. Assim, uma variável ou estrutura isolada no fim do arquivo não constitui programa completo. Uma estrutura continua tendo a forma `typedef struct { ... };`, sem identificador após a chave.

2.2.3 Declarações locais e comandos iniciados por ID

No início do corpo, `ID` pode representar um tipo definido pelo usuário ou o começo de uma expressão. `Body` consome esse prefixo e delega a `BodyId`. Se o próximo token for outro `ID`, a produção reconhece a lista de variáveis; se for uma continuação de expressão, `IdExprRest` completa chamadas, índices e operadores a partir do identificador já consumido. `PrimType` reúne os quatro tipos primitivos; `StmtNoId` e `ExprNoId` tratam os demais inícios. Depois de iniciado o trecho de comandos, a análise segue por `StmtList`. Declarações locais permanecem antes dos comandos; blocos internos contêm comandos. Essa organização aplica novamente a fatoração de (LOUDEN, 2004, seção 4.2.3, p. 162--163): a decisão é adiada até o ponto em que as alternativas se distinguem pelo token corrente. O parser não consulta uma tabela semântica para decidir se o primeiro identificador é um tipo; essa validação permanece fora desta etapa, conforme o enunciado (Disciplina de Compiladores, s.d.b).

2.2.4 Expressões, precedência e associatividade

A produção inicial `Expr -> Expr BinOp Expr` é ambígua e recursiva à esquerda. Foi substituída por níveis. Louden mostra que uma mesma sequência pode gerar árvores com operações em ordens diferentes (LOUDEN, 2004, seção 3.4.1, p. 114--117); em seguida, apresenta a cascata de precedência, com uma regra para cada grupo de operadores (LOUDEN, 2004, seção 3.4.2, p. 118--119). Essa técnica foi escolhida para que a própria gramática determine o agrupamento, em vez de depender de uma correção posterior da AST. A ordem concreta dos níveis segue o enunciado (Disciplina de Compiladores, s.d.b), da menor para a maior precedência:

Nível	Operadores ou construção
`AssignExpr`	`=`
`OrExpr`	`||`
`AndExpr`	`&&`
`EqualityExpr`	`==`, `!=`
`RelExpr`	`<`, `<=`, `>`, `>=`
`AddExpr`	`+`, `-`
`MulExpr`	`*`, `/`, `%`
`UnaryExpr`	`-`, `++`, `!`
`Primary/Atom`	índices, identificadores, chamadas, literais e parênteses

Cada nível consome primeiro o de maior precedência e então sua cauda. Para operadores binários, a cauda recebe o nó acumulado à esquerda: `a-b-c` produz `(a-b)-c`, apesar da recursão à direita da produção auxiliar. A atribuição usa `Assign' -> = AssignExpr | epsilon`, produzindo `a=(b=c)`. Parênteses devolvem a expressão interna, preservando seu agrupamento sem criar nó de pontuação. A passagem do resultado acumulado para a cauda recursiva segue a solução de (LOUDEN, 2004, seção 4.2.3, p. 161--162): eliminar a recursão à esquerda modifica a árvore de derivação, mas a ação de construção deve conservar a associatividade pretendida. A regra de atribuição, por sua vez, aplica a associação à direita explicada no capítulo 3 (LOUDEN, 2004, seção 3.4.2, p. 117--122).

`ID` e `ID(ExprList)` foram fatorados em `ID CallOpt`; `Primary -> Atom Index'` permite aplicar índices sucessivos. A alternativa de comando de chamada foi incorporada a `Expr ;`, que já reconhece chamadas. Não foram acrescentados `for`, sinal negativo unário, inicialização na declaração ou `else` opcional.

2.2.5 Inclusão de blocos vazios

A lista inicial exigia pelo menos um comando. A extensão foi adotada para acompanhar a possibilidade sintática de um bloco vazio em C: no documento N3220, a regra de comando composto é

`{ block-item-list`$_{\texttt{opt}}$ `}`, cuja lista é opcional (ISO/IEC JTC 1/SC 22/WG14, 2024, seção 6.8.3, p. 153). Assim, permitir `{}` e funções com corpo vazio tem uma referência explícita na linguagem de origem. O termo ‘‘vazio’’ descreve o conteúdo sintático do bloco; não envolve implementar ou verificar escopos.

As demais regras continuam sendo as de C-- fornecidas no projeto, inclusive `else` obrigatório e a forma de declaração de estrutura. Para representar a ausência de comandos, introduz-se a alternativa ε e seleciona-se essa alternativa pelo seguidor do não terminal, conforme o tratamento das produções vazias em (LOUDEN, 2004, seção 4.2.2, p. 154). Na versão final:

```
StmtList -> Stmt StmtList | epsilon
Body -> PrimType idList ; Body | ID BodyId
     | StmtNoId StmtList | epsilon
```

A alternativa vazia de `StmtList` permite `{}` em comandos; a de `Body` permite um corpo de função sem comandos. São duas aplicações da mesma extensão. Por exemplo, `int main(){}` deriva por `Program`, `Type ID RestoDecl`, pelo ramo de função de `RestoDecl`, e pelas alternativas vazias de `FormalList`, `Body` e `Program'`. Um `if` ainda exige os dois ramos, mesmo que ambos sejam blocos vazios.

2.2.6 Verificação e gramática resultante

O utilitário <tests/gramatica/analiseLl1.py> calcula anulabilidade, FIRST e FOLLOW até um ponto fixo; para cada produção, preenche a tabela com FIRST da sequência e, quando ela é anulável, com FOLLOW do lado esquerdo. Também procura ciclos de recursão à esquerda, considerando prefixos anuláveis. A execução encontrou 44 não terminais, nenhum ciclo e nenhuma célula com produções concorrentes. O relatório integral pode ser visto executando:

```
python tests/gramatica/analiseLl1.py tests/gramatica/gramaticaCompleta_4.g
```

Isso verifica a propriedade LL(1) da gramática fornecida; isoladamente, não é uma prova formal de equivalência com a inicial. As produções foram comparadas manualmente para avaliar a preservação das construções originais[1]. As transformações descritas e os exemplos experimentais complementam a verificação. O cálculo até estabilização segue os exemplos de FIRST e o algoritmo de FOLLOW de (LOUDEN, 2004, seção 4.3, p. 171--175); o preenchimento da tabela segue a seleção por início e por seguimento das produções anuláveis (LOUDEN, 2004, seção 4.2.2, p. 154). Esses conjuntos foram usados porque uma alternativa vazia não pode ser escolhida para qualquer token: ela só é adequada quando o contexto admite encerrar aquele não terminal.

A gramática final é apresentada no Apêndice B. As transformações foram aplicadas manualmente, com conferência de cada passo. A verificação automática confirma a ausência de conflitos LL(1) e a consistência dos conjuntos FIRST/FOLLOW. A gramática final é usada pelo parser recursivo descendente preditivo implementado.

2.3 Construção da árvore de sintaxe abstrata

A árvore é construída durante o reconhecimento, por ações que instanciam os nós ao obter os componentes de cada construção. A separação entre ações e tipos da árvore segue (APPEL; GINSBURG, 1998, seções 4.1--4.2); a representação por árvores e atributos sintetizados segue (AHO et al., 2007, seções 5.1 e 5.3), atendendo à interface e aos nós equivalentes solicitados (Disciplina de Compiladores, s.d.b).

[1] <linkdrive>

Nós	Conteúdo e construção
`Programa`	Lista ordenada de declarações globais, preenchida por `program`. A classe de cada elemento distingue variáveis, estruturas e funções, em lugar de três listas encadeadas separadas.
`DeclVariavel`, `Parametro`	Tipo e nome textuais, tamanho inteiro opcional. Uma lista de identificadores gera um nó para cada nome.
`DeclStruct`	Lista de campos; não armazena nome de alias, ausente na produção adotada.
`DeclFuncao`, `Bloco`	Função guarda tipo de retorno, nome, parâmetros e corpo. O corpo separa declarações locais de comandos. Blocos vazios produzem listas vazias, não ausência de nó.
`If`, `While`	Condição e comandos filhos; `If` tem ramos `entao` e `senao`.
`Print`, `Readln`, `Return`, `Break`	Lista de argumentos, alvo, valor e comando sem operandos, respectivamente.
`ComandoExpr`	Encapsula uma expressão seguida de ponto e vírgula.
`Id`, `Literal`	Nome ou tipo do token e lexema do literal; não avaliam valores.
`OpBinaria`, `OpUnaria`	Operador e referências aos operandos.
`Atribuicao`, `Indexacao`, `Chamada`	Alvo e valor; base e índice; nome e lista de argumentos, respectivamente.

Todos os nós herdam linha e coluna de `NoAST`. As classes são declaradas em <ast/Nos.py> com `dataclass`; listas Python representam sequências e referências entre objetos representam as arestas. A AST não contém os não terminais auxiliares, vírgulas, delimitadores ou ponto e vírgula. A razão é a distinção entre árvore de derivação e árvore abstrata: em (LOUDEN, 2004, seção 3.3.2, p. 109--110), a AST conserva operações e operandos e elimina informação recuperável da própria hierarquia, como parênteses. O exemplo de `if` com condição e dois ramos, na p. 112, fundamenta os três filhos de `If`.

Para sequências, Louden apresenta ligações entre irmãos como representação de uma quantidade variável de elementos (LOUDEN, 2004, seção 3.3.2, p. 113--114). No projeto, listas Python cumprem esse papel, preservando a ordem e permitindo percorrer parâmetros, argumentos e comandos sem criar um nó de pontuação para cada separador. As classes de dados são uma adaptação dessa representação à linguagem de implementação: explicitam quais filhos e atributos pertencem a cada construção. Guardar a posição de origem permite associar posteriormente os nós a diagnósticos, sem antecipar as verificações semânticas.

Ao reconhecer `a = 1 + 2 * 3`, o nível multiplicativo cria primeiro `OpBinaria(MULT,2,3)`. O aditivo liga esse nó ao literal 1; a atribuição liga o identificador `a` à soma. A função `formalList`, por sua vez, retorna o primeiro parâmetro concatenado aos demais: `return [parametro] + self.formalRest()`. O retorno transporta os nós construídos para `DeclFuncao`; não altera a regra reconhecida. A mesma ideia de atributos retornados é aplicada às listas e expressões. O fundamento é o parser que retorna uma árvore e liga os resultados dos procedimentos filhos, apresentado em (LOUDEN, 2004, seção 4.1.2, p. 150--151) e nas ações de (APPEL; GINSBURG, 1998, seções 4.1--4.2). O livro fundamenta retornar estruturas; a concatenação de listas é a forma escolhida no código para manter a ordem dos parâmetros. Ela não é uma condição do método recursivo: uma implementação iterativa com inserção em lista também poderia construir a mesma AST. O retorno foi usado aqui para acompanhar a composição recursiva de `FormalList/FormalRest`.

A fachada `Parser.analisar(tokens)` devolve `Programa`; o método do símbolo inicial constrói e retorna essa raiz nos caminhos reconhecidos. Em recuperação, preservam-se nós completos já obtidos e a saída identifica a árvore como parcial. Ela não deve ser interpretada como aprovação do programa. A visualização textual percorre os campos e listas das classes; a imagem opcional envia a descrição ao executável `dot` pela entrada padrão.

2.4 Gerenciamento e recuperação de erros

A implementação combina correções locais de um token com sincronização. A escolha evita descartar uma construção inteira quando falta apenas pontuação, mas fornece uma saída quando a correção local não é segura. Louden recomenda diagnóstico próximo à falha, retomada em um ponto provável e prevenção de cascatas e laços infinitos (LOUDEN, 2004, seção 4.5, p. 183); também apresenta FOLLOW e FIRST como candidatos à sincronização e observa que pontuação ausente nem sempre deve provocar descarte (LOUDEN, 2004, seção 4.5.1, p. 184--185). Essas decisões são compatíveis com (APPEL; GINSBURG, 1998, seção 3.5), (AHO et al., 2007, seções 4.1.4 e 4.4.5) e a recuperação mínima prevista no enunciado (Disciplina de Compiladores, s.d.b).

`casar(esperado)` compara o token atual com o esperado. Se forem diferentes e o token seguinte for o esperado, descarta somente o token excedente e continua o casamento. Caso contrário, se o token atual estiver entre os seguidores configurados para inserção, registra a ausência e retorna um token sintético sem consumir a entrada. Isso permite inserir, por exemplo, `]`, `)` ou `;`. O token sintético não é informado como terminal efetivamente casado. A consulta ao token seguinte pertence à recuperação; a escolha das produções válidas continua preditiva LL(1).

Quando a correção local não se aplica, `falhar` registra o diagnóstico e lança `SyntaxError`. Os pontos de recuperação chamam `sincronizar` com FOLLOW e a posição inicial da construção. O algoritmo descarta tokens até um limite ou início seguro, considera ponto e vírgula e EOF, acompanha a profundidade de chaves e pode preservar `else` para o comando que o espera. O controle de avanço evita repetir indefinidamente a mesma posição. Essa estratégia pode perder a construção inválida, mas mantém as construções recuperáveis seguintes. O descarte até um token de sincronização adapta o modo pânico de (LOUDEN, 2004, seção 4.5.1, p. 184). A contagem de chaves e a preservação de `else` são ajustes ao contexto da gramática deste projeto: evitam retomar em um bloco interno ou consumir o delimitador do comando externo. São heurísticas de recuperação justificadas pela estrutura reconhecida, não garantias de reconstruir a intenção original do autor.

Os erros são armazenados em uma lista de `ErroSintatico`, contendo mensagem, token encontrado com linha e coluna, conjunto imutável de esperados e descrição da correção local quando houver. `obterErros()` devolve uma cópia da lista; `limpar()` prepara outra análise. Na interface de linha de comando, os diagnósticos são emitidos ao registrar, em `stderr`, enquanto o rastro e a AST seguem para `stdout`. A lista preserva os diagnósticos para consulta e para determinar o resultado da análise; a cópia devolvida impede que o chamador modifique acidentalmente o estado interno. A emissão separada segue o contrato de entrada e saída do trabalho (Disciplina de Compiladores, s.d.b), e a distinção entre notificar erro e continuar reconhecendo segue (LOUDEN, 2004, seção 4.5, p. 183).

Caso ocorra erros léxicos, a execução informa os erros e encerra antes de chamar o parser. Embora o enunciado proponha continuar o parsing após falhas léxicas, o fluxo adotado mantém o bloqueio combinado para o projeto. Trata-se de uma política entre etapas, independente das produções da gramática; não é uma segunda extensão da linguagem. Sua justificativa no projeto é impedir que uma sequência de tokens já incompleta ou inválida seja tratada como entrada aprovada para a etapa seguinte.

A execução usa `python main.py arquivo.cmm -arvore`. O código de retorno é 0 quando a análise termina sem erros, 1 para erros léxicos ou sintáticos e 2 para falhas de leitura ou visualização. A opção `-imagem arvore.png` requer Graphviz[2] disponível.

2.5 Resultados experimentais

Foram usadas as dez entradas disponíveis em <tests/parser>, chamando a mesma função de entrada da interface com a opção `-arvore`. Foram aceitas seis entradas sem diagnóstico. Quatro entradas inválidas terminaram com retorno 1 e sete diagnósticos sintáticos no total. Para cada entrada,

[2] <https://graphviz.org/download/>

as listagens seguintes apresentam o código e a árvore resultante; nos casos inválidos, também apresentam integralmente os diagnósticos. O rastro completo de não terminais e casamentos podem ser obtidos na execução.

2.5.1 `valido_atribuicao_expressao`

A atribuição a uma soma de indexações é aceita sintaticamente. A AST conserva a expressão no campo alvo, permitindo que a futura etapa semântica rejeite um l-value inválido.

Programa 2: Entrada do caso.

```
int main() {
    int a[2], b[2], valor;
    a[0] + b[1] = valor;
    return 0;
}
```

Programa 3: Árvore impressa em stdout.

```
Árvore sintática abstrata:
Programa
`-- declaracoes[0]: DeclFuncao (tipoRetorno=int, nome=main)
    `-- corpo: Bloco
        |-- declaracoes[0]: DeclVariavel (tipo=int, nome=a, tamanho=2)
        |-- declaracoes[1]: DeclVariavel (tipo=int, nome=b, tamanho=2)
        |-- declaracoes[2]: DeclVariavel (tipo=int, nome=valor)
        |-- comandos[0]: ComandoExpr
        | `-- expr: Atribuicao
        | |-- alvo: OpBinaria (operador=PLUS)
        | | |-- esquerda: Indexacao
        | | | |-- vetor: Id (nome=a)
        | | | `-- indice: Literal (tipo=NUM_INT, valor=0)
        | | `-- direita: Indexacao
        | | |-- vetor: Id (nome=b)
        | | `-- indice: Literal (tipo=NUM_INT, valor=1)
        | `-- valor: Id (nome=valor)
        `-- comandos[1]: Return
            `-- valor: Literal (tipo=NUM_INT, valor=0)
```

2.5.2 `valido_blocos_vazios`

Função e blocos de comandos vazios são aceitos e representados por listas de comandos vazias. O else permanece obrigatório.

Programa 4: Entrada do caso.

```
int vazia() {}

int main() {
    while (true) {}
    if (false) {} else {}
}
```

Programa 5: Árvore impressa em stdout.

```
Árvore sintática abstrata:
Programa
|-- declaracoes[0]: DeclFuncao (tipoRetorno=int, nome=vazia)
```

```
| `-- corpo: Bloco
`-- declaracoes[1]: DeclFuncao (tipoRetorno=int, nome=main)
    `-- corpo: Bloco
        |-- comandos[0]: While
        | |-- condicao: Literal (tipo=TRUE, valor=true)
        | `-- corpo: Bloco
        `-- comandos[1]: If
            |-- condicao: Literal (tipo=FALSE, valor=false)
            |-- entao: Bloco
            `-- senao: Bloco
```

2.5.3 `valido_chamadas_vetores`

Exercita parâmetros, argumentos, acesso a vetores e chamadas em expressões. A AST mantém as listas na ordem original.

Programa 6: Entrada do caso.

```
int soma(int a, int b) {
    return a + b;
}

int main() {
    int valores[4], indice;
    indice = 1;
    valores[indice] = soma(indice, 2);
    print(valores[indice], soma(1, 2));
    return valores[indice];
}
```

Programa 7: Árvore impressa em stdout.

```
Árvore sintática abstrata:
Programa
|-- declaracoes[0]: DeclFuncao (tipoRetorno=int, nome=soma)
| |-- parametros[0]: Parametro (tipo=int, nome=a)
| |-- parametros[1]: Parametro (tipo=int, nome=b)
| `-- corpo: Bloco
| `-- comandos[0]: Return
| `-- valor: OpBinaria (operador=PLUS)
| |-- esquerda: Id (nome=a)
| `-- direita: Id (nome=b)
`-- declaracoes[1]: DeclFuncao (tipoRetorno=int, nome=main)
    `-- corpo: Bloco
        |-- declaracoes[0]: DeclVariavel (tipo=int, nome=valores, tamanho=4)
        |-- declaracoes[1]: DeclVariavel (tipo=int, nome=indice)
        |-- comandos[0]: ComandoExpr
        | `-- expr: Atribuicao
        | |-- alvo: Id (nome=indice)
        | `-- valor: Literal (tipo=NUM_INT, valor=1)
        |-- comandos[1]: ComandoExpr
        | `-- expr: Atribuicao
        | |-- alvo: Indexacao
        | | |-- vetor: Id (nome=valores)
        | | `-- indice: Id (nome=indice)
        | `-- valor: Chamada (nome=soma)
        | |-- argumentos[0]: Id (nome=indice)
        | `-- argumentos[1]: Literal (tipo=NUM_INT, valor=2)
        |-- comandos[2]: Print
```

```
       | |-- argumentos[0]: Indexacao
       | | |-- vetor: Id (nome=valores)
       | | `-- indice: Id (nome=indice)
       | `-- argumentos[1]: Chamada (nome=soma)
       | |-- argumentos[0]: Literal (tipo=NUM_INT, valor=1)
       | `-- argumentos[1]: Literal (tipo=NUM_INT, valor=2)
       `-- comandos[3]: Return
           `-- valor: Indexacao
               |-- vetor: Id (nome=valores)
               `-- indice: Id (nome=indice)
```

2.5.4 `valido_controle`

Exercita while, if/else, blocos, break, leitura, impressão e retorno. A hierarquia da árvore mostra o aninhamento dos comandos.

Programa 8: Entrada do caso.

```
int main() {
    int indice;
    indice = 0;

    while (indice < 3) {
        if (indice == 1) {
            print("meio", indice);
        } else {
            readln(indice);
        }
        indice = indice + 1;
    }

    return indice;
}
```

Programa 9: Árvore impressa em stdout.

```
Árvore sintática abstrata:
Programa
`-- declaracoes[0]: DeclFuncao (tipoRetorno=int, nome=main)
    `-- corpo: Bloco
        |-- declaracoes[0]: DeclVariavel (tipo=int, nome=indice)
        |-- comandos[0]: ComandoExpr
        | `-- expr: Atribuicao
        | |-- alvo: Id (nome=indice)
        | `-- valor: Literal (tipo=NUM_INT, valor=0)
        |-- comandos[1]: While
        | |-- condicao: OpBinaria (operador=LESS_THAN)
        | | |-- esquerda: Id (nome=indice)
        | | `-- direita: Literal (tipo=NUM_INT, valor=3)
        | `-- corpo: Bloco
        | |-- comandos[0]: If
        | | |-- condicao: OpBinaria (operador=EQUALS)
        | | | |-- esquerda: Id (nome=indice)
        | | | `-- direita: Literal (tipo=NUM_INT, valor=1)
        | | |-- entao: Bloco
        | | | `-- comandos[0]: Print
        | | | |-- argumentos[0]: Literal (tipo=LITERAL, valor="meio")
        | | | `-- argumentos[1]: Id (nome=indice)
        | | `-- senao: Bloco
```

```
       | | `-- comandos[0]: Readln
       | | `-- alvo: Id (nome=indice)
       | `-- comandos[1]: ComandoExpr
       | `-- expr: Atribuicao
       | |-- alvo: Id (nome=indice)
       | `-- valor: OpBinaria (operador=PLUS)
       | |-- esquerda: Id (nome=indice)
       | `-- direita: Literal (tipo=NUM_INT, valor=1)
       `-- comandos[2]: Return
           `-- valor: Id (nome=indice)
```

2.5.5 `valido_declaracoes`

Reconhece campos de estrutura, variáveis globais, parâmetros e funções. Os tipos e nomes são preservados nos nós; a estrutura não possui alias após a chave.

Programa 10: Entrada do caso.

```
typedef struct {
    int codigo;
    float saldo;
    char categoria;
    bool ativo;
};

int totais[3], contador;

float media(float a, float b) {
    return (a + b) / 2;
}

int main() {
    int indice;
    indice = 0;
    return indice;
}
```

Programa 11: Árvore impressa em stdout.

```
Árvore sintática abstrata:
Programa
|-- declaracoes[0]: DeclStruct
| |-- campos[0]: DeclVariavel (tipo=int, nome=codigo)
| |-- campos[1]: DeclVariavel (tipo=float, nome=saldo)
| |-- campos[2]: DeclVariavel (tipo=char, nome=categoria)
| `-- campos[3]: DeclVariavel (tipo=bool, nome=ativo)
|-- declaracoes[1]: DeclVariavel (tipo=int, nome=totais, tamanho=3)
|-- declaracoes[2]: DeclVariavel (tipo=int, nome=contador)
|-- declaracoes[3]: DeclFuncao (tipoRetorno=float, nome=media)
| |-- parametros[0]: Parametro (tipo=float, nome=a)
| |-- parametros[1]: Parametro (tipo=float, nome=b)
| `-- corpo: Bloco
| `-- comandos[0]: Return
| `-- valor: OpBinaria (operador=DIV)
| |-- esquerda: OpBinaria (operador=PLUS)
| | |-- esquerda: Id (nome=a)
| | `-- direita: Id (nome=b)
| `-- direita: Literal (tipo=NUM_INT, valor=2)
`-- declaracoes[4]: DeclFuncao (tipoRetorno=int, nome=main)
```

```
    `-- corpo: Bloco
        |-- declaracoes[0]: DeclVariavel (tipo=int, nome=indice)
        |-- comandos[0]: ComandoExpr
        | `-- expr: Atribuicao
        | |-- alvo: Id (nome=indice)
        | `-- valor: Literal (tipo=NUM_INT, valor=0)
        `-- comandos[1]: Return
            `-- valor: Id (nome=indice)
```

2.5.6 `valido_precedencia`

A multiplicação fica abaixo da soma na AST; os parênteses alteram o agrupamento. A atribuição encadeada associa à direita. Os nós lógicos distinguem AND e OR, e os relacionais aparecem no nível previsto.

Programa 12: Entrada do caso.

```
int main() {
    int a, b, c;
    bool ok;

    a = 1 + 2 * 3;
    b = (1 + 2) * 3;
    c = a = b + 4;
    ok = !false || true && a < b + c == false;

    return a;
}
```

Programa 13: Árvore impressa em stdout.

```
Árvore sintática abstrata:
Programa
`-- declaracoes[0]: DeclFuncao (tipoRetorno=int, nome=main)
    `-- corpo: Bloco
        |-- declaracoes[0]: DeclVariavel (tipo=int, nome=a)
        |-- declaracoes[1]: DeclVariavel (tipo=int, nome=b)
        |-- declaracoes[2]: DeclVariavel (tipo=int, nome=c)
        |-- declaracoes[3]: DeclVariavel (tipo=bool, nome=ok)
        |-- comandos[0]: ComandoExpr
        | `-- expr: Atribuicao
        | |-- alvo: Id (nome=a)
        | `-- valor: OpBinaria (operador=PLUS)
        | |-- esquerda: Literal (tipo=NUM_INT, valor=1)
        | `-- direita: OpBinaria (operador=MULT)
        | |-- esquerda: Literal (tipo=NUM_INT, valor=2)
        | `-- direita: Literal (tipo=NUM_INT, valor=3)
        |-- comandos[1]: ComandoExpr
        | `-- expr: Atribuicao
        | |-- alvo: Id (nome=b)
        | `-- valor: OpBinaria (operador=MULT)
        | |-- esquerda: OpBinaria (operador=PLUS)
        | | |-- esquerda: Literal (tipo=NUM_INT, valor=1)
        | | `-- direita: Literal (tipo=NUM_INT, valor=2)
        | `-- direita: Literal (tipo=NUM_INT, valor=3)
        |-- comandos[2]: ComandoExpr
        | `-- expr: Atribuicao
        | |-- alvo: Id (nome=c)
        | `-- valor: Atribuicao
```

```
      | |-- alvo: Id (nome=a)
      | `-- valor: OpBinaria (operador=PLUS)
      | |-- esquerda: Id (nome=b)
      | `-- direita: Literal (tipo=NUM_INT, valor=4)
      |-- comandos[3]: ComandoExpr
      | `-- expr: Atribuicao
      | |-- alvo: Id (nome=ok)
      | `-- valor: OpBinaria (operador=OR)
      | |-- esquerda: OpUnaria (operador=NEG)
      | | `-- operando: Literal (tipo=FALSE, valor=false)
      | `-- direita: OpBinaria (operador=AND)
      | |-- esquerda: Literal (tipo=TRUE, valor=true)
      | `-- direita: OpBinaria (operador=EQUALS)
      | |-- esquerda: OpBinaria (operador=LESS_THAN)
      | | |-- esquerda: Id (nome=a)
      | | `-- direita: OpBinaria (operador=PLUS)
      | | |-- esquerda: Id (nome=b)
      | | `-- direita: Id (nome=c)
      | `-- direita: Literal (tipo=FALSE, valor=false)
      `-- comandos[4]: Return
          `-- valor: Id (nome=a)
```

2.5.7 `erro_sintatico_delimitadores`

Produz quatro diagnósticos: duas inserções de colchete, uma de parêntese e uma de ponto e vírgula. A árvore parcial registra as construções recuperadas.

Programa 14: Entrada do caso.

```
int main() {
    int valores[3;
    print(valores[1;
    return 0
}
```

Programa 15: Diagnósticos em stderr.

```
Erro sintático na linha 2, coluna 18: Falta ']'
Encontrado: SEMICOLON (';')
Esperado: CLOSE_BRACKETS
Recuperação: Token CLOSE_BRACKETS inserido antes de SEMICOLON

Erro sintático na linha 3, coluna 20: Falta ']'
Encontrado: SEMICOLON (';')
Esperado: CLOSE_BRACKETS
Recuperação: Token CLOSE_BRACKETS inserido antes de SEMICOLON

Erro sintático na linha 3, coluna 20: Falta ')'
Encontrado: SEMICOLON (';')
Esperado: CLOSE_PARENTHESES
Recuperação: Token CLOSE_PARENTHESES inserido antes de SEMICOLON

Erro sintático na linha 5, coluna 1: Falta ';'
Encontrado: CLOSE_BRACES ('}')
Esperado: SEMICOLON
Recuperação: Token SEMICOLON inserido antes de CLOSE_BRACES
```

Programa 16: Árvore impressa em stdout.

```
Árvore sintática abstrata (parcial: houve erros na análise):
Programa
`-- declaracoes[0]: DeclFuncao (tipoRetorno=int, nome=main)
    `-- corpo: Bloco
        |-- declaracoes[0]: DeclVariavel (tipo=int, nome=valores, tamanho=3)
        |-- comandos[0]: Print
        | `-- argumentos[0]: Indexacao
        | |-- vetor: Id (nome=valores)
        | `-- indice: Literal (tipo=NUM_INT, valor=1)
        `-- comandos[1]: Return
            `-- valor: Literal (tipo=NUM_INT, valor=0)
```

2.5.8 erro_sintatico_else

A ausência de else é diagnosticada. A recuperação permite alcançar o retorno posterior; a aceitação do programa não é sinalizada.

Programa 17: Entrada do caso.

```
int main() {
    if (x1 <= 32) b = 10;
    return 0;
}
```

Programa 18: Diagnósticos em stderr.

```

Erro sintático na linha 3, coluna 5: Token inesperado
Encontrado: RETURN ('return')
Esperado: ELSE
```

Programa 19: Árvore impressa em stdout.

```
Árvore sintática abstrata (parcial: houve erros na análise):
Programa
`-- declaracoes[0]: DeclFuncao (tipoRetorno=int, nome=main)
    `-- corpo: Bloco
        `-- comandos[0]: Return
            `-- valor: Literal (tipo=NUM_INT, valor=0)
```

2.5.9 erro_sintatico_struct_com_nome

O identificador após a chave não pertence à produção inicial nem à final. A correção local descarta o ID excedente e a função seguinte permanece na árvore parcial.

Programa 20: Entrada do caso.

```
typedef struct {
    int codigo;
    float saldo;
} Conta;

int main() {}
```

Programa 21: Diagnósticos em stderr.

```
Erro sintático na linha 4, coluna 3: Token inesperado antes de ';'
Encontrado: ID ('Conta')
Esperado: SEMICOLON
Recuperação: Token ID descartado
```

Programa 22: Árvore impressa em stdout.

```
Árvore sintática abstrata (parcial: houve erros na análise):
Programa
|-- declaracoes[0]: DeclStruct
| |-- campos[0]: DeclVariavel (tipo=int, nome=codigo)
| `-- campos[1]: DeclVariavel (tipo=float, nome=saldo)
`-- declaracoes[1]: DeclFuncao (tipoRetorno=int, nome=main)
    `-- corpo: Bloco
```

2.5.10 `erro_sintatico_token_extra`

O identificador excedente antes do ponto e vírgula é descartado. O retorno é preservado e o processo termina com erro.

Programa 23: Entrada do caso.

```
int main() {
    return 1 extra;
}
```

Programa 24: Diagnósticos em stderr.

```
Erro sintático na linha 2, coluna 14: Token inesperado antes de ';'
Encontrado: ID ('extra')
Esperado: SEMICOLON
Recuperação: Token ID descartado
```

Programa 25: Árvore impressa em stdout.

```
Árvore sintática abstrata (parcial: houve erros na análise):
Programa
`-- declaracoes[0]: DeclFuncao (tipoRetorno=int, nome=main)
    `-- corpo: Bloco
        `-- comandos[0]: Return
            `-- valor: Literal (tipo=NUM_INT, valor=1)
```

2.5.11 Rastro do reconhecimento

O trecho a seguir é o início da saída real do caso de blocos vazios. Ele mostra a visita aos não terminais e os terminais casados, inclusive a escolha da produção vazia de `Body`. A AST é impressa após o rastro completo.

Programa 26: Trecho do rastro

```
Program
Type
Match: INT
Match: ID.vazia
```

```
RestoDecl
Match: LPARENT
FormalList
Match: RPARENT
Match: OPEN_BRACES
Body
Match: CLOSE_BRACES
Program'
Program
Type
Match: INT
Match: ID.main
RestoDecl
Match: LPARENT
FormalList
Match: RPARENT
Match: OPEN_BRACES
Body
StmtNoId
Match: WHILE
```

2.5.12 Discussão dos resultados

As árvores obtidas distinguem declarações, comandos e expressões, sem materializar os não terminais introduzidos apenas para fatoração. Os casos inválidos demonstram tanto a correção local quanto a retomada após uma construção incorreta. Em todos eles, a indicação de árvore parcial e o retorno 1 impedem confundir recuperação com sucesso.

A propriedade LL(1) foi verificada separadamente pelo cálculo dos conjuntos e da tabela. A aceitação sintática não garante correção semântica ou execução: compatibilidade de tipos, escopos e l-values pertencem à próxima etapa.

3 Considerações finais

A segunda entrega implementa o reconhecimento preditivo, a construção da AST em memória e a recuperação sintática. A separação entre fluxo de tokens, motor, registros de erro e nós permite reutilizar a árvore nas próximas etapas, conforme as interfaces e ações de (APPEL; GINSBURG, 1998, seções 1.1 e 4.1--4.2), a análise descendente de (AHO et al., 2007, seção 4.4) e a organização solicitada (Disciplina de Compiladores, s.d.b).

A conversão remove prefixos concorrentes e recursão à esquerda, organiza a precedência e conserva as restrições descritas para a linguagem. Blocos vazios constituem a única extensão deliberada da gramática. A atribuição entre expressões é mantida para posterior validação semântica. A política acordada de interromper o fluxo antes do parser quando há erros léxicos foi registrada separadamente.

Como recursos adicionais, estão disponíveis a impressão hierárquica da AST, a exportação de imagem por Graphviz e a consulta programática aos erros armazenados. A validação experimental confirmou os resultados dos dez casos apresentados e a ausência de conflitos LL(1) na gramática final.

As decisões de representação e transformação se apoiam também em Louden: abstração da árvore e cascata de precedência no capítulo 3 (LOUDEN, 2004, seções 3.3.2 e 3.4.2, p. 109--114 e 118--119); tradução das produções em procedimentos e retorno de subárvores no capítulo 4 (LOUDEN, 2004, seções 4.1.1--4.1.2, p. 144--151). A inclusão de blocos vazios acompanha a lista opcional de itens do comando composto e seu uso como corpo de função em C (ISO/IEC JTC 1/SC 22/WG14, 2024,

seções 6.8.3 e 6.9.2, p. 153 e 159). As técnicas gerais foram adaptadas às estruturas de Python e às regras de C--, mantendo identificáveis a fonte conceitual, a aplicação no código e as decisões específicas do projeto.

Apêndices

A Gramática Inicial

Programa 27: Gramática inicial.

```
Program -> FunctionDecl Program |
               TypeDecl Program |
               VarDecl Program |
               FunctionDecl

TypeDecl --> typdedef struct {Type IdList ; VarDecl} ; TypeDecl | epsilon

VarDecl -> Type idList ; VarDecl | epsilon

IdList -> ID Array |
       IdList, ID Array

Array -> [ NUM ] | epsilon

FunctionDecl -> Type ID ( FormalList ) { VarDecl StmList }

FormalList -> Type ID Array FormalRest | epsilon

FormalRest -> , Type ID Array FormalRest | epsilon

Type -> int | float | bool | ID | char

StmtList -> Stmt | Stmt StmtList

Stmt -> if ( Expr ) Stmt else Stmt |
               while ( Expr ) Stmt |
               break ; |
               print ( ExprList ) ; |
               readln ( Expr ) ; |
               return Expr ;|
               { StmtList } |
               ID ( ExprList ) ; |
               Expr ;

ExprList -> epsilon | ExprListTail

ExprListTail -> Expr | Expr , ExprListTail

Expr -> Primary |
               UnaryOp Expr |
               Expr BinOp Expr |
               Expr = Expr

Primary -> ID | NUM | LITERAL |
               'ASCII '|
               ( Expr ) |
               ID ( ExpreList ) |
               Expr [ Expr ] |
               true | false

UnaryOp -> -- | ++ | !

BinOp ->  == | < | <= |
```

```
                    >= | > | != | + | - |
                    || | * | / | % | &&
```

B Gramática Transformada

Programa 28: Gramática transformada.

```
Program -> TypeDeclOne Program | Type ID RestoDecl

TypeDeclOne -> typedef struct { Type idList ; VarDecl } ;

RestoDecl -> ( FormalList ) { Body } Program' | Array idList' ; Program

Program' -> Program | ε

VarDecl -> Type idList ; VarDecl | ε

Body -> PrimType idList ; Body | ID BodyId | StmtNoId StmtList | ε

BodyId -> idList ; Body | IdExprRest ; StmtList

IdExprRest -> CallOpt Index' Mul' Add' Rel' Equality' And' Or' Assign'

PrimType -> int | float | bool | char

StmtNoId -> if ( Expr ) Stmt else Stmt | while ( Expr ) Stmt | break ; | print ( ExprList
    ) ; | readln ( Expr ) ; | return Expr ; | { StmtList } | ExprNoId ;

ExprNoId -> UnaryOp UnaryExpr Mul' Add' Rel' Equality' And' Or' Assign' | AtomNoId Index'
    Mul' Add' Rel' Equality' And' Or' Assign'

AtomNoId -> NUM_INT | NUM_FLOAT | LITERAL | ASCII | ( Expr ) | true | false

idList -> ID Array idList'

idList' -> , ID Array idList' | ε

Array -> [ NUM_INT ] | ε

FormalList -> Type ID Array FormalRest | ε

FormalRest -> , Type ID Array FormalRest | ε

Type -> int | float | bool | ID | char

StmtList -> Stmt StmtList | ε

Stmt -> if ( Expr ) Stmt else Stmt | while ( Expr ) Stmt | break ; | print ( ExprList ) ;
    | readln ( Expr ) ; | return Expr ; | { StmtList } | Expr ;

ExprList -> ε| ExprList'

ExprList' -> Expr RestoExprList

RestoExprList -> , ExprList' | ε

Expr -> AssignExpr
```

```
AssignExpr -> OrExpr Assign'

Assign' -> = AssignExpr | ε

OrExpr -> AndExpr Or'

Or' -> || AndExpr Or' | ε

AndExpr -> EqualityExpr And'

And' -> && EqualityExpr And' | ε

EqualityExpr -> RelExpr Equality'

Equality' -> == RelExpr Equality' | != RelExpr Equality' | ε

RelExpr -> AddExpr Rel'

Rel' -> < AddExpr Rel' | <= AddExpr Rel' | > AddExpr Rel' | >= AddExpr Rel' | ε

AddExpr -> MulExpr Add'

Add' -> + MulExpr Add' | - MulExpr Add' | ε

MulExpr -> UnaryExpr Mul'

Mul' -> * UnaryExpr Mul' | / UnaryExpr Mul' | % UnaryExpr Mul' | ε

UnaryExpr -> UnaryOp UnaryExpr | Primary

Primary -> Atom Index'

Index' -> [ Expr ] Index' | ε

Atom -> ID CallOpt | NUM_INT | NUM_FLOAT | LITERAL | ASCII | ( Expr ) | true | false

CallOpt -> ( ExprList ) | ε

UnaryOp -> -- | ++ | !
```

Referências

AHO, A. V. et al. *Compilers: Principles, Techniques, and Tools*. 2. ed. Boston: Addison-Wesley, 2007.

APPEL, A. W.; GINSBURG, M. *Modern Compiler Implementation in C*. Cambridge: Cambridge University Press, 1998.

Disciplina de Compiladores. *Projeto e implementação de compilador: parte 1*. s.d. Enunciado fornecido pela disciplina.

Disciplina de Compiladores. *Projeto e implementação de um compilador: parte 2*. s.d. Enunciado fornecido pela disciplina: documentação mínima, especificações do parser, recuperação de erros e árvore de sintaxe abstrata.

ISO/IEC JTC 1/SC 22/WG14. *Information technology --- Programming languages --- C: N3220 working draft*. 2024. Disponível em: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3220.pdf>. Acesso em: 12 set. 2026.

LOUDEN, K. C. *Compiladores: Princípios e Práticas*. [S.l.]: Cengage Learning Editores, 2004.